

Laboratorio 03: Preprocesamiento y Visualización de Datos

Fundamentos de Inteligencia Artificial

6 de marzo de 2026

1. INSTRUCCIONES

1.1. OBJETIVOS

Desarrollar competencias básicas para:

1. Lectura y exploración de datos utilizando **pandas**.
2. Limpieza y transformación de datos utilizando **pandas**.
3. Visualización básica de datos utilizando **matplotlib** y **seaborn**.
4. Análisis y preprocesamiento de datos para la formulación de hipótesis de modelos.

1.2. ENTREGABLE

Reglas para el envío de los entregables:

- **Forma de envío:**

Este laboratorio se debe enviar únicamente a través de la plataforma [Moodle](#) en la actividad definida.

- **Archivo de entrega:**

Incluyan en un archivo **.zip** los archivos correspondientes al laboratorio, revise [How to cite in Jupyter Notebooks](#) para citar cualquier recurso utilizado y ver la estructura del proyecto.

- **Nomenclatura para nombrar el archivo de entrega:**

El archivo deberá ser renombrado en mayúsculas con el siguiente formato:
`L03-{color_grupo}-IAIA-{semestre_academico}.zip`

Ejemplos:

```
L03-AZUL-IAIA-2026-1.zip
L03-ROJO-IAIA-2026-1.zip
```

1.3. Herramientas del laboratorio

En este laboratorio trabajaremos con tres bibliotecas fundamentales del ecosistema de Python para ciencia de datos:

- **pandas**: Biblioteca para manipulación y análisis de datos. Proporciona estructuras de datos como **Series** (unidimensional) y **DataFrame** (bidimensional, tipo tabla) que facilitan la lectura, limpieza, transformación y exploración de conjuntos de datos. Es el estándar en Python para trabajar con datos tabulares. [Documentación oficial](#).
- **matplotlib**: Biblioteca base de visualización en Python. Permite crear gráficos estáticos como líneas, barras, dispersión, histogramas, entre otros. Es altamente configurable y sirve como fundamento sobre el cual se construyen otras bibliotecas de visualización. [Documentación oficial](#).
- **seaborn**: Biblioteca de visualización construida sobre **matplotlib**. Simplifica la creación de gráficos comunes (distribuciones, relaciones entre variables, comparaciones por categorías) con menos código y estilos más atractivos por defecto. [Documentación oficial](#).

Revise cuando lo requiera (Russell & Norvig, 2021) [3]

2. I. COMPRENDIENDO

En esta sección aprenderá los conceptos básicos de **pandas**, **matplotlib** y **seaborn** a través de ejemplos simples y ejercicios guiados.

2.1. Convenciones de Importación

La comunidad Python ha adoptado una serie de convenciones de nomenclatura para los módulos de uso común:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Esto significa que cuando ve `pd.DataFrame`, se trata de una referencia a la clase **DataFrame** en **pandas**, o cuando ve `plt.show()`, es una referencia a la función **show** de **matplotlib**. Esto se hace porque se considera una mala práctica en el desarrollo de software Python importar todo (`from pandas import *`) de un paquete grande.

Por ahora, solo importaremos **numpy** y **pandas**. Más adelante, cuando lleguemos a la sección de visualización, importaremos **matplotlib** y **seaborn**.

```
[ ]: import pandas as pd
```

2.2. Pandas: Estructuras de Datos

Para empezar a utilizar **pandas**, debe comprender sus dos estructuras de datos principales: **Series** y **DataFrame**. Ambas se construyen a partir de estructuras básicas de Python como **listas** y **diccionarios**.

2.2.1. Series

Una **Series** es un arreglo unidimensional que contiene una secuencia de valores y un arreglo asociado de etiquetas, denominado **index**. Se puede crear a partir de una **lista** o un **diccionario** de

Python.

Creación desde una lista:

```
[ ]: # Desde una lista, el index se asigna automáticamente (0, 1, 2, ...)  
series_lista = pd.Series([10, 20, 30, 40])  
series_lista
```

```
[ ]: # Desde una lista, con un index personalizado  
series_index = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])  
series_index
```

Creación desde un diccionario:

```
[ ]: # Desde un diccionario, las llaves se convierten en el index  
series_dict = pd.Series({"manzana": 3, "banana": 5, "naranja": 2})  
series_dict
```

Ejercicio: Manipulando Series Tiene los siguientes datos de población:

Estado	Población
Ohio	35000
Texas	71000
Oregon	16000
Utah	5000

1. Cree la **Series** utilizando un **diccionario** de Python a partir de los datos previos y observe el resultado.
2. Convierta la **Series** nuevamente a un diccionario de Python. Investigue qué método de **Series** se usa para esto.
3. Use `pd.isna()` y `pd.notna()` sobre su **Series**. ¿Qué devuelven? ¿Qué significa NaN en el contexto de pandas?
4. Asigne un nombre a la **Series** (“Population”) y al index (“Estado”). Investigue los atributos `.name` e `.index.name`.

Referencia: [pandas Series](#) y [Pandas Tutorial](#)

```
[ ]: # **COMPLETE**
```

2.2.2. DataFrame

Un **DataFrame** representa una tabla de datos con filas y columnas. Cada columna puede tener un tipo de dato diferente. Se puede considerar como un **diccionario de Series** que comparten el mismo índice.

La forma más común de construir un **DataFrame** es a partir de un **diccionario de listas** de igual longitud:

```
[ ]: data = {
    "state": ["Ohio", "Ohio", "Ohio", "Nevada", "Nevada", "Nevada"],
    "year": [2000, 2001, 2002, 2001, 2002, 2003],
    "pop": [1.5, 1.7, 3.6, 2.4, 2.9, 3.2],
}
frame = pd.DataFrame(data)
frame
```

Ejercicio: Manipulando DataFrame

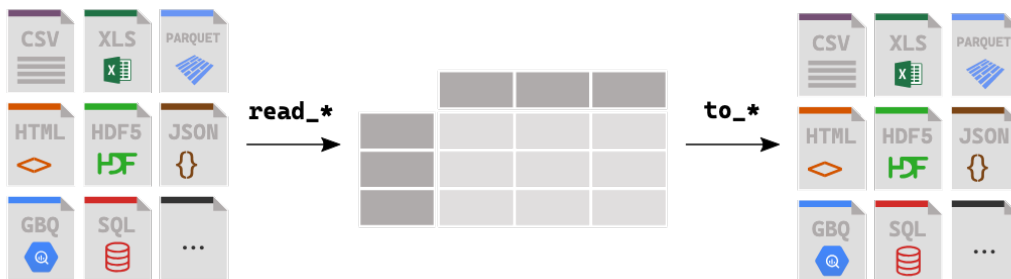
1. Use las funciones `head()` y `tail()` sobre `frame`. ¿Qué hacen? ¿Qué parámetro aceptan?
2. Use `describe()` sobre `frame`. ¿Qué información proporciona? ¿Qué columnas incluye en el resumen y por qué?
3. Cree un nuevo `DataFrame` a partir de `data` usando el parámetro `columns`. Agregue columnas que no existan en `data` y observe qué sucede.
4. Extraiga los nombres de las columnas de su `DataFrame`. Investigue cómo hacerlo.
5. Modifique las nuevas columnas agregadas con un valor constante.
6. Cree una `Series` con nuevos datos y agréguela como columna “*learning*” al `DataFrame`.
7. Cree una columna “*eastern*” que sea `True` si el estado es Ohio y `False` en caso contrario.
8. Elimine la columna “*pop*” usando la palabra reservada `del` de Python. Observe el resultado.

Referencia: [pandas DataFrame](#) y [Pandas Tutorial](#)

```
[ ]: # **COMPLETE**
```

2.2.3. Lectura de Datos desde Archivos

En la práctica, rara vez se crean `DataFrames` manualmente. Lo más común es **leer datos desde archivos** en diferentes formatos. `pandas` ofrece múltiples funciones de lectura según el tipo de fuente:



La función más usada es `pd.read_csv()`:

```
# Desde un archivo local
df = pd.read_csv("ruta/al/archivo.csv")

# Desde una URL
df = pd.read_csv("https://ejemplo.com/datos.csv")
```

```
# Especificando la primera columna como índice
df = pd.read_csv("archivo.csv", index_col=0)
```

Ejercicio: Cargando y explorando un dataset

1. Cargue el conjunto de datos `data/pokemon.csv` usando `pd.read_csv()`. Use el parámetro `index_col=0` para que la primera columna sea el índice.
2. Observe los primeros 5 registros con `head()` y los últimos 5 con `tail()`.
3. Use `info()` para ver información general del `DataFrame`: ¿Cuántas filas y columnas tiene? ¿Hay datos faltantes? ¿Qué tipos de datos tiene cada columna?
4. Use `describe()` para obtener el resumen estadístico. Explique qué representa cada fila: `count`, `mean`, `std`, `min`, `25%`, `50%`, `75%`, `max`.
5. Use `shape` y `dtypes` para verificar dimensiones y tipos de datos.

```
[ ]: # **COMPLETE**
```

2.2.4. Selección de Datos en DataFrame

Aquí se muestra un resumen de las formas más comunes de seleccionar datos en un `DataFrame` `df`:



Tipo	Descripción
<code>df[columna]</code>	Selecciona una columna o secuencia de columnas.
<code>df.loc[filas]</code>	Selecciona filas por etiqueta (nombre del índice).
<code>df.loc[:, columnas]</code>	Selecciona columnas por etiqueta .
<code>df.loc[filas, columnas]</code>	Selecciona filas y columnas por etiqueta .
<code>df.iloc[filas]</code>	Selecciona filas por posición (número entero).
<code>df.iloc[:, columnas]</code>	Selecciona columnas por posición .
<code>df.iloc[filas, columnas]</code>	Selecciona filas y columnas por posición .

Ejercicio: Selección de datos Usando el `DataFrame` de Pokémon cargado anteriormente:

1. Seleccione la columna `Attack` usando `df['Attack']`. ¿Qué tipo de objeto devuelve?
2. Seleccione las columnas `Attack` y `Defense` al mismo tiempo.
3. Use `loc` para seleccionar el Pokémon con índice 1 (Bulbasaur). ¿Qué información obtiene?
4. Use `iloc` para seleccionar las primeras 3 filas y las columnas del 1 al 4.

5. Seleccione todos los Pokémon cuyo **Attack** sea mayor a 100. ¿Cuántos son?

```
[ ]: # **COMPLETE**
```

2.2.5. Manejo de datos faltantes

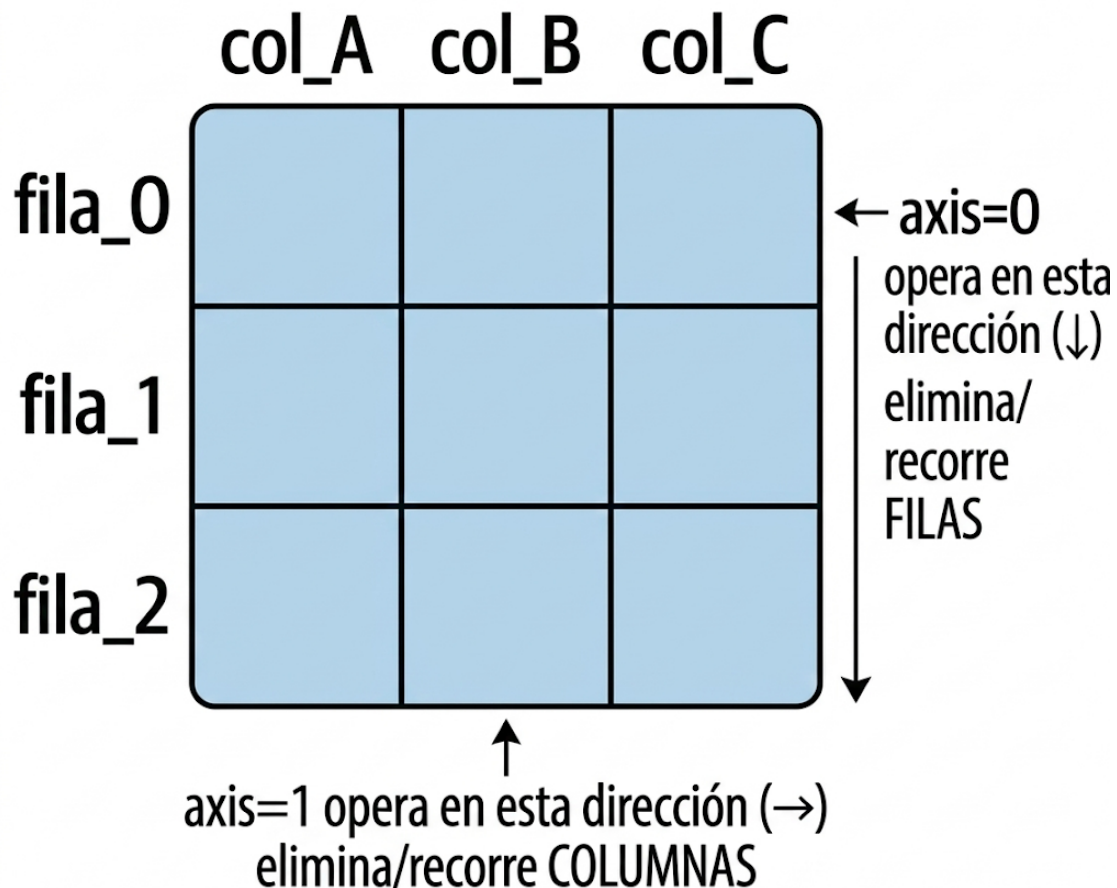
Los datos faltantes son comunes en conjuntos de datos reales. **pandas** representa los datos faltantes como **NaN** (*Not a Number*) y ofrece funciones para detectarlos, eliminarlos o rellenarlos.

Las funciones principales son:

- `df.isnull()` y `pd.isna()`: Detecta valores faltantes.
- `df.dropna()`: Elimina filas o columnas con valores faltantes.
- `df.fillna(valor)`: Rellena valores faltantes con un valor dado.

El parámetro axis Muchas funciones de **pandas** (como `dropna()`, `sum()`, `mean()`, `apply()`, `drop()`) reciben un parámetro **axis** que indica la **dirección** en la que se aplica la operación. Este concepto es fundamental y se usa en toda la biblioteca:

- **axis=0** (o `axis='index'`): opera sobre las **filas** (recorre de arriba hacia abajo). Es el valor por defecto.
- **axis=1** (o `axis='columns'`): opera sobre las **columnas** (recorre de izquierda a derecha).



Ejemplos:

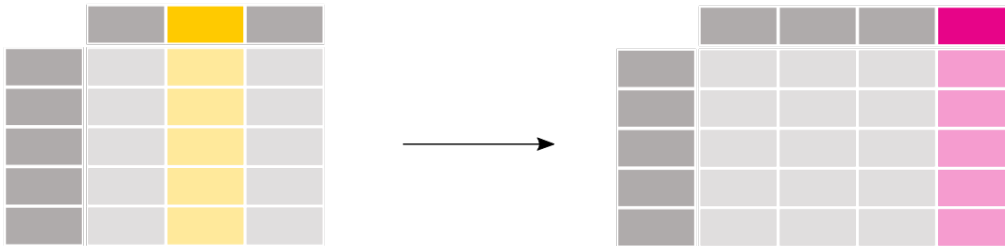
- `df.dropna(axis=0)` → elimina las **filas** que tengan algún valor faltante.
- `df.dropna(axis=1)` → elimina las **columnas** que tengan algún valor faltante.
- `df.sum(axis=0)` → suma cada **columna** (recorriendo las filas hacia abajo).
- `df.sum(axis=1)` → suma cada **fila** (recorriendo las columnas hacia la derecha).

Ejercicio: Datos faltantes

1. Cree un `DataFrame` con datos faltantes (use `np.nan` o `None`) y aplique `dropna()` y `fillna()` para observar cómo se comportan.
2. Con `dropna()`: elimine datos faltantes solo de las filas (`axis=0`) y luego solo de las columnas (`axis=1`). ¿Cuál es la diferencia?
3. Con `fillna()`: rellene los datos faltantes con un valor constante (por ejemplo, 0 o la media de la columna).

```
[ ]: # **COMPLETE**
```

2.2.6. Transformación de datos



Eliminando duplicados

1. Observe el `DataFrame` creado a partir de `data` (abajo).
2. Use `duplicated()` para identificar filas duplicadas y `drop_duplicates()` para eliminarlas. No modifique el `DataFrame` original.
3. Agregue una columna `data["v1"] = range(7)` y aplique `drop_duplicates()` especificando el subconjunto de columnas `["k1"]`. Observe el resultado.

```
[ ]: data = pd.DataFrame({"k1": ["one", "two"] * 3 + ["two"], "k2": [1, 1, 2, 3, 3, 4, 4]})
```

```
[ ]: # **COMPLETE**
```

Transformación usando `map()` y `replace()` En muchos conjuntos de datos necesitará transformar valores de una columna. Antes de ver cómo hacerlo en `pandas`, repasemos la función `map()` de Python.

`map()` en Python básico:

La función built-in `map(función, iterable)` aplica una función a cada elemento de un iterable (lista, tupla, etc.) y devuelve un iterador con los resultados:

map() en pandas:

- **Con un diccionario:** transforma cada valor buscándolo en el diccionario.
- **Con una función:** aplica la función a cada elemento de la **Series**.

```
[ ]: data = pd.DataFrame(  
    data={  
        "food": ["bacon", "pulled pork", "bacon", "pastrami", "corned beef",  
↪ "bacon", "pastrami", "honey ham", "nova lox"],  
        "ounces": [4, 3, 12, 6, 7.5, 8, 3, 5, 6],  
    }  
)
```

```
[ ]: meat_to_animal = {
    "bacon": "pig",
    "pulled pork": "pig",
    "pastrami": "cow",
    "corned beef": "cow",
    "honey ham": "pig",
    "nova lox": "salmon",
}
```

1. Use `map()` con el diccionario `meat_to_animal` para crear una nueva columna `animal` en el `DataFrame`.

2. Cree una función `get_animal(meat: str) -> str` que devuelva el tipo de animal dado un tipo de carne. Use esta función con `map()` para crear la columna `animal`.

```
[ ]: # **COMPLETE**
```

Reemplazando valores con `replace()` `replace()` permite sustituir valores específicos de forma directa, sin necesidad de definir un mapeo completo:

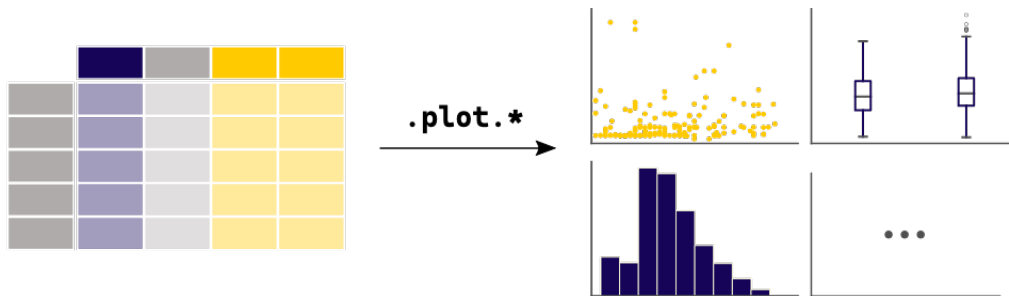
```
[ ]: data = pd.Series([1., -999., 2., -999., -1000., 3.])
```

Ejercicio:

1. Los valores `-999` representan datos faltantes. Reemplácelos por `np.nan` usando `replace()`.
2. Ahora tanto `-999` como `-1000` son datos faltantes. Reemplace ambos por `np.nan` en una sola operación usando `replace()`.

```
[ ]: # **COMPLETE**
```

2.3. Matplotlib: Visualización Básica



Antes de usar `seaborn`, es importante conocer los fundamentos de `matplotlib`, la biblioteca base de visualización en Python. `matplotlib` le da control total sobre los elementos de un gráfico: ejes, títulos, colores, tamaños, etc.

Importemos la biblioteca de visualización:

```
[ ]: import matplotlib.pyplot as plt
```

2.3.1. Conceptos básicos de `matplotlib`

En `matplotlib`, un gráfico se compone de:

- **Figure:** El lienzo o contenedor principal del gráfico.
- **Axes:** El área donde se dibujan los datos (un **Figure** puede tener múltiples **Axes**).

Veamos ejemplos de los tres tipos de gráficos más comunes:

Gráfico de línea — `plt.plot()`: conecta puntos con una línea, útil para datos secuenciales.

```
[ ]: # Gráfico de línea
x = [1, 2, 3, 4, 5]
```

```

y = [2, 4, 1, 8, 5]

plt.figure(figsize=(8, 4))
plt.plot(x, y, marker='o')
plt.title("Gráfico de Línea")
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.show()

```

Gráfico de barras — `plt.bar()`: compara cantidades entre categorías.

```

[ ]: # Gráfico de barras
categorías = ["A", "B", "C", "D"]
valores = [15, 30, 10, 25]

plt.figure(figsize=(8, 4))
plt.bar(categorías, valores, color="steelblue")
plt.title("Gráfico de Barras")
plt.xlabel("Categorías")
plt.ylabel("Valores")
plt.show()

```

Gráfico de dispersión — `plt.scatter()`: muestra la relación entre dos variables numéricas.

```

[ ]: # Gráfico de dispersión
import numpy as np

rng = np.random.default_rng(42)
x = rng.standard_normal(50)
y = x + rng.standard_normal(50) * 0.5

plt.figure(figsize=(8, 4))
plt.scatter(x, y, color="coral")
plt.title("Gráfico de Dispersión")
plt.xlabel("Variable X")
plt.ylabel("Variable Y")
plt.show()

```

Múltiples subgráficos — `plt.subplots()`: permite crear varios gráficos en una sola figura.

```

[ ]: # Múltiples subgráficos en una sola figura
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Subgráfico 1: histograma
datos = rng.standard_normal(200)
axes[0].hist(datos, bins=20, color="skyblue", edgecolor="black")
axes[0].set_title("Histograma")
axes[0].set_xlabel("Valor")
axes[0].set_ylabel("Frecuencia")

```

```
# Subgráfico 2: gráfico de línea
x = range(10)
y = [i ** 2 for i in x]
axes[1].plot(x, y, marker='s', color="tomato")
axes[1].set_title("Línea:  $y = x^2$ ")
axes[1].set_xlabel("x")
axes[1].set_ylabel("y")

plt.tight_layout()
plt.show()
```

Ejercicio: Gráficos básicos con matplotlib Cargue el dataset de Pokémon si no lo tiene cargado:

```
pokemon = pd.read_csv("data/pokemon.csv", index_col=0)
```

1. Cree un **histograma** de la columna **Attack** usando `plt.hist()`. Agregue título y etiquetas a los ejes.
2. Cree un **gráfico de dispersión** de **Attack** vs **Defense** usando `plt.scatter()`. ¿Observa alguna relación?
3. Cree una figura con **2 subgráficos** lado a lado usando `plt.subplots(1, 2, figsize=(12, 5))`. En el primero dibuje un histograma de **HP** y en el segundo un histograma de **Speed**.
4. ¿Qué función usa para ajustar el tamaño de la figura? ¿Y para rotar las etiquetas del eje x?

```
[ ]: # **COMPLETE**
```

2.4. Seaborn: Visualización de Datos

seaborn se construye sobre **matplotlib** y facilita la creación de gráficos con menos código. Mientras **matplotlib** requiere configurar manualmente muchos detalles, **seaborn** ofrece funciones de alto nivel que generan visualizaciones informativas automáticamente.

A continuación exploraremos los tipos de gráficos más comunes de forma sistemática, organizados según el **tipo de variable** que representan.

```
[ ]: import seaborn as sns

# Cargar el dataset de Pokémon
pokemon = pd.read_csv("data/pokemon.csv", index_col=0)
pokemon.head()
```

```
[ ]: pokemon.describe()
```

2.4.1. Tipos de variables y análisis

Antes de visualizar datos, es importante entender los **tipos de variables** y cómo se analizan:

Variables según su tipo

- **Numéricas** (cuantitativas): representan cantidades medibles. Ejemplos: `Attack`, `Defense`, `Speed`, `HP`.
- **Catóricas** (cualitativas): representan grupos o categorías. Ejemplos: `Type 1`, `Generation`, `Legendary`.

Análisis univariado vs bivariado

- **Univariado**: analiza **una sola variable** a la vez. El objetivo es entender su distribución (¿cómo se reparten los valores? ¿hay valores atípicos? ¿cuál es el valor más frecuente?).

Tipo de variable	Gráfico recomendado	¿Qué muestra?
Numérica	<code>histplot</code> , <code>kdeplot</code>	Distribución de valores, forma, sesgo
Catórica	<code>countplot</code>	Frecuencia de cada categoría

- **Bivariado**: analiza la **relación entre dos variables**. El objetivo es descubrir si existe alguna asociación o patrón entre ellas.

Combinación	Gráfico recomendado	¿Qué muestra?
Numérica vs Numérica	<code>scatterplot</code> , <code>lmpplot</code>	Correlación, tendencias
Catórica vs Numérica	<code>boxplot</code> , <code>violinplot</code>	Distribución por grupo
Catórica vs Católica	<code>heatmap</code> (sobre <code>crosstab</code>)	Frecuencia cruzada

En las siguientes secciones exploraremos cada combinación con ejemplos prácticos.

2.4.2. Variable numérica univariada: `histplot`

El histograma muestra la **distribución** de una variable numérica, es decir, cómo se reparten los valores en diferentes rangos.

1. Cree un histograma de `Attack` usando `sns.histplot(data=pokemon, x='Attack')`.
2. Agregue una curva de densidad con `kde=True`. ¿Qué información adicional aporta?
3. Cree un histograma de `Speed`. ¿La distribución es simétrica o sesgada?

```
[ ]: # **COMPLETE**
```

2.4.3. Variable catórica univariada: `countplot`

El gráfico de conteo muestra **cuántos elementos** hay en cada categoría de una variable catórica.

1. Cree un gráfico de conteo de `Type 1` usando `sns.countplot(data=pokemon, x="Type 1")`.
2. Rote las etiquetas del eje x con `plt.xticks(rotation=45)` para mejor legibilidad.
3. ¿Cuál es el tipo de Pokémon más común? ¿Cuál es el menos común?

```
[ ]: # **COMPLETE**
```

2.4.4. Numérica vs numérica: scatterplot

El gráfico de dispersión muestra la **relación entre dos variables numéricas**. Cada punto representa una observación.

1. Cree un gráfico de dispersión de Attack vs Defense usando `sns.scatterplot(data=pokemon, x='Attack', y='Defense')`. ¿Observa alguna relación?
2. Coloree los puntos según Generation usando el parámetro `hue="Generation"`. ¿Nota diferencias entre generaciones?
3. Use `sns.lmplot(data=pokemon, x='Attack', y='Defense', fit_reg=False, hue='Generation')` para obtener un resultado similar. ¿Qué pasa si cambia `fit_reg=True`?

```
[ ]: # **COMPLETE**
```

2.4.5. Categórica vs numérica: boxplot

El diagrama de cajas compara la **distribución de una variable numérica** entre diferentes categorías. Muestra resúmenes de los datos.

1. Cree un gráfico de cajas comparando Attack por Type 1 usando `sns.boxplot(data=pokemon, x='Type 1', y='Attack')`. Use `plt.figure(figsize=(12, 6))` y rote las etiquetas.
2. ¿Cuál tipo de Pokémon tiene el ataque más fuerte (mediana más alta)? ¿Cuál el más débil?
3. ¿Qué representan los puntos individuales fuera de los “bigotes” del diagrama de cajas?

```
[ ]: # **COMPLETE**
```

2.4.6. Categórica vs categórica: heatmap sobre tabla de contingencia

Para ver la relación entre dos variables categóricas, se puede crear una **tabla de contingencia** (conteo cruzado) y visualizarla con un mapa de calor.

1. Cree una tabla de contingencia entre Type 1 y Generation usando `pd.crosstab(pokemon["Type 1"], pokemon["Generation"])`.
2. Visualícela con `sns.heatmap()` usando `annot=True` y `cmap="YlOrRd"`.
3. ¿Qué combinación de tipo y generación tiene más Pokémon?

```
[ ]: # **COMPLETE**
```

2.4.7. Resumen de gráficos y personalización

seaborn incluye temas de estilo que permiten cambiar la apariencia de los gráficos. El tema pre-determinado es **darkgrid**.

1. Cree un gráfico de cajas general con **todas las columnas numéricas** del dataset de Pokémon, usando `plt.figure(figsize=(10, 6))`. Observando las escalas, ¿qué columnas tienen rangos muy diferentes al resto? ¿Cuáles no aportan información útil para comparar?
2. Elimine del **DataFrame** las columnas que identificó como no informativas usando `drop()`. Grafique de nuevo el boxplot sin esas columnas y explique por qué las eliminó.

3. Cambie el estilo a `whitegrid` con `sns.set_style("whitegrid")` y cree un gráfico de violines (`sns.violinplot()`) comparando `Type 1` con `Attack`. ¿Qué información adicional da el `violinplot` respecto al `boxplot`?

Siéntase libre de explorar la [galería oficial de Seaborn](#) como referencia.

```
[ ]: # **COMPLETE**
```

3. II. EXPLORANDO

En esta sección aplicará lo aprendido de forma **guiada**, siguiendo un proceso paso a paso de análisis y preprocesamiento de datos. Trabajaremos con el dataset de Pokémon para cubrir el ciclo completo:

Cargar → Explorar → Visualizar → Limpiar → Transformar → Visualizar → Concluir

3.1. Paso 1: Carga y exploración inicial

1. Cargue el dataset `data/pokemon.csv` con `index_col=0`.
2. Use `info()` y `describe()` para tener una visión general.
3. ¿Cuántos datos faltantes hay en cada columna? Use `isnull().sum()`.
4. ¿Cuántos valores únicos tiene cada columna categórica? Use `nunique()` o `value_counts()`.

```
[ ]: # **COMPLETE**
```

3.2. Paso 2: Visualización exploratoria inicial

Antes de limpiar o transformar, es fundamental **entender los datos visualmente**.

1. Cree un `histplot` para cada variable numérica relevante (`HP`, `Attack`, `Defense`, `Speed`, `Sp. Atk`, `Sp. Def`). Puede usar un bucle `for` con `plt.subplots()`.
2. Cree un `countplot` para `Type 1`. ¿Hay tipos desbalanceados?
3. Cree un `boxplot` general con todas las variables numéricas. ¿Detecta valores atípicos?
4. Cree un `heatmap` de correlación usando `pokemon.corr(numeric_only=True)` con `annot=True`. ¿Qué variables están más correlacionadas?

```
[ ]: # **COMPLETE**
```

3.3. Paso 3: Limpieza de datos

1. Identifique las columnas con datos faltantes. ¿Cuál tiene más?
2. Para `Type 2`: ¿qué significa un valor faltante? ¿Tiene sentido eliminarlo o rellenarlo? Justifique su decisión.
3. Verifique si hay filas duplicadas con `duplicated().sum()`.
4. Verifique si hay valores inconsistentes en las columnas categóricas.

```
[ ]: # **COMPLETE**
```

3.4. Paso 4: Transformación de datos

1. Cree una columna `Total_Stats` sumando `HP`, `Attack`, `Defense`, `Sp. Atk`, `Sp. Def` y `Speed`. Compare con la columna `Total` existente.

2. Cree una columna `Attack_Category` que clasifique el ataque:
 - `Attack < 50` → “Bajo”
 - `50 <= Attack < 100` → “Medio”
 - `Attack >= 100` → “Alto”
 Use `pd.cut()` o una función con `map()`.
3. Transforme `Legendary` de `True/False` a `1/0` usando `astype(int)`.
4. Elimine columnas que no aporten al análisis. Justifique.

```
[ ]: # **COMPLETE**
```

3.5. Paso 5: Visualización posterior a la transformación

Ahora que los datos están limpios y transformados, verifique visualmente los resultados.

1. Cree un `countplot` de `Attack_Category`. ¿Cómo se distribuyen los Pokémon?
2. Cree un `boxplot` de `Total_Stats` por `Attack_Category`.
3. Cree un `scatterplot` de `Attack` vs `Defense`, coloreado por `Attack_Category`.
4. Compare con las visualizaciones del Paso 2. ¿Qué insights nuevos obtiene?

```
[ ]: # **COMPLETE**
```

3.6. Paso 6: Conclusiones

Escriba un resumen con:

1. Las características más importantes del dataset.
2. Las relaciones encontradas entre variables.
3. Los problemas de calidad de datos y cómo los resolvió.
4. Las transformaciones realizadas y su justificación.

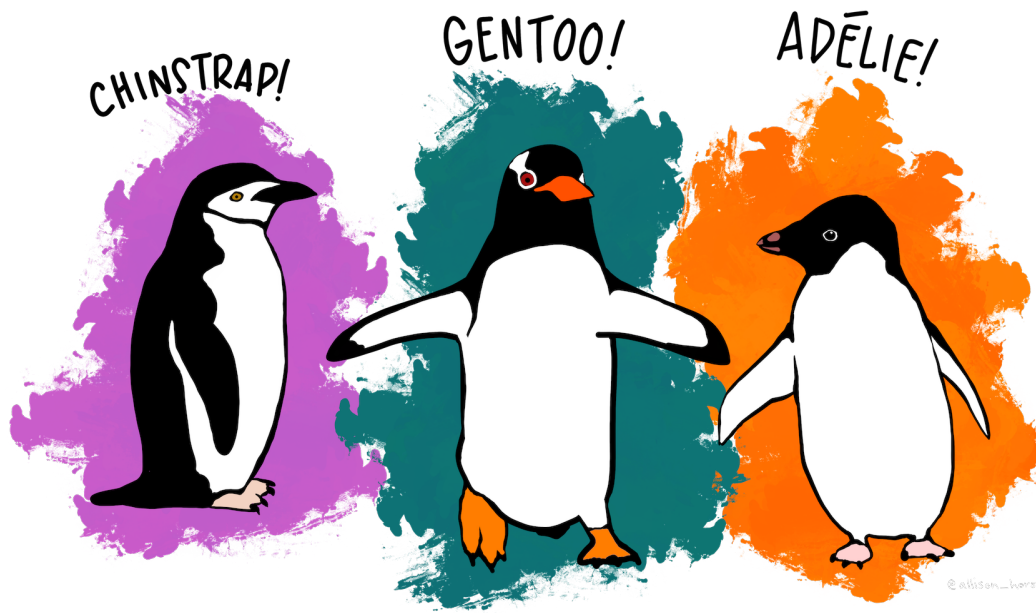
4. III. APLICANDO

En esta sección aplicará de forma **autónoma** todo lo aprendido. Trabjará con tres datasets, cada uno representando un tipo de problema distinto. Para cada uno debe:

1. Realizar el proceso completo de **análisis y preprocesamiento**.
2. **Proponer una hipótesis del modelo** que podría resolver el problema.

4.1. Predicción Numérica (Regresión)

Dataset: `data/penguins.csv` [2]



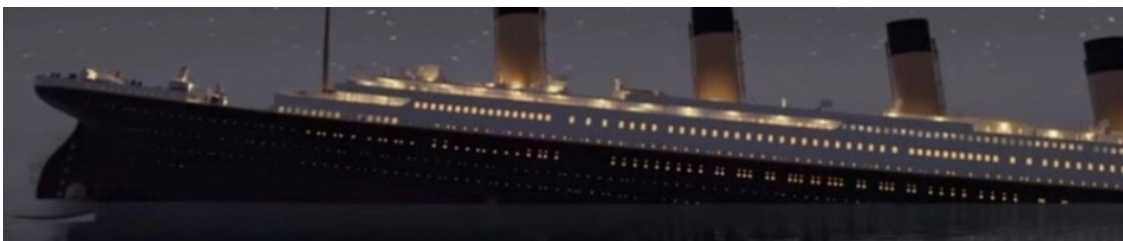
Objetivo: Predecir la masa corporal (`body_mass_g`) de un pingüino a partir de sus otras características.

1. **Carga y exploración:** Use `read_csv()`, `info()`, `describe()`, `head()`.
2. **Visualización exploratoria:** Al menos 3 gráficos que muestren relaciones con `body_mass_g`.
3. **Limpieza:** Identifique y maneje datos faltantes. Justifique.
4. **Transformación:** Realice las transformaciones necesarias.
5. **Hipótesis del modelo:** ¿Qué variables son más importantes para predecir `body_mass_g`? ¿Qué tipo de modelo sugeriría? Justifique.

```
[ ]: # **COMPLETE**
```

4.2. Clasificación

Dataset: `data/titanic.csv` [1]



Objetivo: Predecir si un pasajero sobrevivió (`survived`) al naufragio del Titanic.

1. **Carga y exploración:** Use `read_csv()`, `info()`, `describe()`, `head()`.
2. **Visualización exploratoria:** Al menos 3 gráficos que muestren factores de supervivencia.
3. **Limpieza:** Identifique y maneje datos faltantes. Justifique.

4. **Transformación:** Codifique variables categóricas si es necesario.
5. **Hipótesis del modelo:** ¿Qué variables predicen mejor la supervivencia? ¿Qué modelo sugeriría? Justifique.

```
[ ]: # **COMPLETE**
```

4.3. Agrupamiento (Clustering)

Dataset: data/pokemon.csv



Objetivo: Agrupar los Pokémon en categorías significativas basándose en sus datos de combate.

1. **Carga y exploración:** Seleccione las columnas numéricas de datos (HP, Attack, Defense, Sp. Atk, Sp. Def, Speed).
2. **Visualización exploratoria:** Al menos 3 gráficos para identificar posibles grupos naturales (pairplot, scatterplot, heatmap).
3. **Limpieza y transformación:** Maneje datos faltantes. Considere si necesita normalizar las variables.
4. **Hipótesis del modelo:** ¿Cuántos grupos naturales cree que existen? ¿Qué variables distinguen mejor los grupos? ¿Qué modelo sugeriría? Justifique.

```
[ ]: # **COMPLETE**
```

5. IV. RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)
2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
3. ¿Cuál consideran fue el mayor logro? ¿Por qué?
4. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
5. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Referencias

- [1] Will Cukierski. Titanic - machine learning from disaster. <https://kaggle.com/competitions/titanic>, 2012. Kaggle.
- [2] Allison Marie Horst, Alison Presmanes Hill, and Kristen B Gorman. *palmerpenguins: Palmer Archipelago (Antarctica) penguin data*, 2020. R package version 0.1.0.
- [3] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach, Global Edition*. Pearson Education, 2021.